AWS OIDC flow: how a pipeline gets into AWS with no stored secret

From zero to reading the ACN-2026-31070 advisory control by control. The same parts carry any AWS authentication case.

1. Identity and credentials

AWS is a set of services inside an **account**. Every request to it has to be signed by someone. That someone is an **IAM identity**.

- An **IAM user** has long-term credentials: an **access key ID** and a **secret access key**. They live until someone deletes them. Anyone who copies them *is* that user.
- An **IAM role** has no password and no keys. Anyone allowed to **assume** it borrows it for a while and gets **temporary credentials**. [1]
- **STS** (Security Token Service) is the AWS service that hands out those temporary credentials. They are three values: access key ID, secret key, and a **session token**. They expire; the default for web identity is one hour, and it can be set from 15 minutes up to the role's maximum of 1 to 12 hours. [4]

AWS advises against long-term credentials in applications outside AWS. [1] The old way here keeps a key in an Azure DevOps variable group. The new way keeps nothing.

Does not apply when: a workload truly cannot use a role. Then keys are the exception, and must be reviewed. [9]

2. OIDC and tokens

OpenID Connect (OIDC) is a standard way for one system (the **identity provider**, IdP) to prove who someone is to another system. The proof is a **JWT**, a signed token with fields called **claims**. [1]

The three claims you will audit:

Claim	Meaning	In this case
`iss`	who issued the token (the IdP URL)	`https://vstoken.dev.azure.com/{OrganizationGUID}` [3]
`aud`	who the token is meant for	`api://AzureADTokenExchange` (fixed for Azure DevOps) [3]
`sub`	who the token is about	`sc://{Org}/{Project}/{ServiceConnection}` [3]

AWS checks the token's signature using public keys from the IdP's `jwks_uri`, found in the IdP's discovery document at `/.well-known/openid-configuration`. [2] A token past its `exp` time is still accepted for five minutes to allow for clock drift. [1]

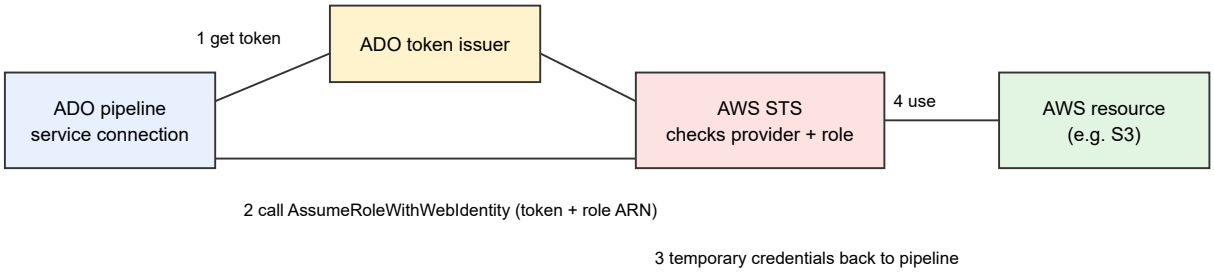
An example token body (values shortened):

```
{ "iss": "https://vstoken.dev.azure.com/8b91...",
  "aud": "api://AzureADTokenExchange",
  "sub": "sc://WBG-ORG/ITSPL/<service-connection-name>" }
```

`iss` names the organisation; `sub` names the service connection inside it.

3. The flow

The pipeline never holds a secret. It asks Azure DevOps for a token, and swaps that token at AWS.



1. The service connection gets an OIDC token from Azure DevOps.
2. It calls STS `AssumeRoleWithWebIdentity` with that token and the **role ARN**. This call needs no AWS credentials. [4]
3. STS checks the token against the **IAM OIDC provider** and the role's **trust policy**. If both pass, it returns temporary credentials.
4. The pipeline signs its AWS calls with them.

The captured evidence shows this: "Getting OIDC Token", then "Assuming role via OIDC Token", then a role ARN `arn:aws:sts::706569143085:assumed-role/WBG-ADO-ORG-SERVICE-CONNECTION-TEST/aws-vsts-tools`, then a successful `aws s3 ls`. `assumed-role` proves a role was used, not a user. `aws-vsts-tools` is the default session name. [3]

4. The IAM OIDC provider

This is an object in the AWS account that says "I trust tokens from this issuer." [2] You create it with the issuer URL as **Provider URL** and `api://AzureADTokenExchange` as **Audience**. [3]

Audit view: one provider per Azure DevOps organisation, URL ending in that organisation's GUID. A provider for a test, personal, or generic org is a finding (control A1). The provider ARN is the anchor for your inventory (control C1).

5. The role trust policy

A role has **two** policy kinds. Do not mix them.

- **Trust policy**: who may assume the role. Authentication.
- **Permission policy**: what the role may do once assumed. Authorization.

This is a good trust policy. [3]

```
{ "Effect": "Allow",
  "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-provider/vstoken.dev.azure.com/<OrgGUID>"
},
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": { "StringEquals": {
     "vstoken.dev.azure.com/<OrgGUID>:aud": "api://AzureADTokenExchange",
     "vstoken.dev.azure.com/<OrgGUID>:sub": "sc://<Org>/<Project>/<ServiceConnection>" } } }
```

Read it in four parts: **Principal** (only this provider), **Action** (only web-identity assume, nothing broader), **aud** (exact), **sub** (exact, one service connection). The condition key name starts with the provider URL; that is how AWS knows which provider's `aud` and `sub` you mean. [3]

Second example, one thing changed. Replace `StringEquals` on `sub` with `StringLike` and the value `sc://WBG-ORG/*`. Now every service connection in the organisation can assume the role. Wildcards are the usual failure, and the advisory bans them (A2, A4). `StringEquals` is an exact match; wildcards are allowed only in the `Like` operators. [8]

Trap: if a condition key is missing from the request, the match is false. For `NotLike` operators it is true instead. [8]

The prod rule (A6): the role used for production must have its own role and its own `sub`. A dev service connection must not match it.

6. Permissions and least privilege

Once assumed, AWS checks what the role may do. A request is allowed only if a policy allows it, and an **explicit deny** always wins. [5]

Layers can only narrow:

- **Permissions boundary**: allowed = identity policy ∩ boundary. [5]
- **Organization SCP (service control policy)**: the action must also be allowed by the SCP. [5]
- **Session policy** passed at assume time: result is the intersection with the role's policy; it can never grant more. [4]

Least privilege means the role lists only the actions and resource ARNs the pipeline needs (A5). A `*` on action or resource is the red flag.

Common confusion: login worked, job gets `AccessDenied`. The trust policy let the pipeline in; the permission policy did not allow the action.

P1 asks who can edit the provider, trust policies and permissions. A pipeline role that can edit its own trust policy can widen its own access.

7. Pipeline side and secrets

The parts are the service connection, the old keys, and the logs. This only works if all three are clean; one stored key defeats the design.

- The **service connection** in Azure DevOps stores the role ARN and the "use OIDC" option. No keys. [3]
- Migration means: switch on OIDC, set the role, and delete the old access key and secret. [3] The advisory goes further: revoke the old keys in AWS too, and leave no fallback in variable groups, YAML, scripts, repos, or agent config (S1).
- Temporary keys and tokens must not appear in logs, artifacts, output variables or caches (S2).
- Changes to the role ARN, the service-connection reference, OIDC enablement and the credential duration need change control and independent review (D1). The duration is the variable `aws.rolecredential.maxduration`, 900 to 43200 seconds. [3]
- Test the negative cases before go-live: wrong audience, wrong subject, unauthorised connection, removed trust (D2).

8. Logging, monitoring, response

AWS **CloudTrail** records IAM and STS API calls, including who, from which IP, and when. A **trail** keeps them long term in an S3 bucket. [6]

What the advisory wants recorded (L1, L2): every `AssumeRoleWithWebIdentity`, success and failure, with enough context to match it to a role and a pipeline run; and every create, change or delete of the provider, roles, trust policies and permission policies.

What it wants watched (M1): repeated failures, unapproved `sub` or `aud`, unexpected assumptions, unauthorised changes.

What it wants ready (R1): a tested way to cut off the federation (narrow the `sub`, disable the connection or the role), then review STS events and revoke leftover keys.

9. Audit method

For each control do four things: **what does it protect, where is the evidence, what does pass look like, what is the finding if it fails.** Open the object; a statement is not evidence.

ID	Control (short)	Open this	Pass	Fail
A1	Provider uses the org-specific issuer	IAM > Identity providers	one URL with the org GUID	generic, test or personal org
A2	`aud` exact, `StringEquals`	role trust policy	`api://AzureADTokenExchange`, exact	wildcard or `Like`
A3	Only `sts:AssumeRoleWithWebIdentity`	trust policy Action	that one action, that one Principal	broader action or other principals
A4	`sub` exact, WBG format	trust policy Condition	one `sc://` value	wildcard, missing sub
A5	Least privilege	role permission policies	named actions and ARNs	`*` or unused rights
A6	Prod role not reachable by dev/test	prod role `sub`	prod connection only	shared role or shared sub
P1	Admin path restricted	who holds `iam:*` on these objects	approved admins only	pipeline role can edit itself
S1	Old keys removed and revoked	IAM user keys, ADO variable groups, YAML	none present, keys deleted	key still active or stored
S2	No secrets in logs	pipeline logs, artifacts	no token or temp key	token printed on error
C1	Inventory, obsolete cleanup	provider and role list	list matches reality	stale provider or role
D1	Change control	pipeline repo history	reviewed changes	direct, unreviewed edits
D2	Pre-prod test with negatives	test records	success plus four denials	success only
L1	STS events logged	CloudTrail	events seen, success and fail	gaps or no trail
L2	IAM changes logged	CloudTrail	create/modify/delete visible	missing
M1	Detection configured	SIEM or alarm rules	rules for the listed events	none
R1	Response documented and tested	runbook, test record	documented and exercised	written but untested

Read the evidence document the same way. The IDs above are labels for this page; the advisory does not number its controls. The captured pipeline log ("OIDC issuer", "Assuming role via OIDC Token", the `assumed-role` ARN) shows only the pipeline's view. It hints at A1 and S1, but the trust policy, the role's permissions and CloudTrail must be opened in AWS.

10. Adjacent AWS authentication

Once you hold the parts above, other cases are the same pieces in different places.

- **Humans**: federate through an IdP to get temporary credentials; AWS recommends IAM Identity Center for central access. [9]
- **Other CI/CD**: GitHub Actions uses the same OIDC pattern with its own issuer. [1]
- **Apps with users**: Amazon Cognito brokers OIDC for sign-up and sign-in. [1]
- **Cross-account and service roles**: a role assumed by another account or an AWS service. [10]

For any new case ask: who is the IdP, what proves the caller, what restricts which caller, what can it do after, where is it logged?

References

1. https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_providers_oidc.html : OIDC federation, JWT exchange, clock window.
2. https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_providers_create_oidc.html : provider prerequisites.
3. https://aws.amazon.com/blogs/modernizing-with-aws/how-to-federate-into-aws-from-azure-devops-using-openid-connect/ : Azure DevOps to AWS steps, trust policy, sub format.
4. https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRoleWithWebIdentity.html : duration, session policies.
5. https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html : deny wins, boundaries, SCPs.
6. https://docs.aws.amazon.com/IAM/latest/UserGuide/cloudtrail-integration.html : CloudTrail for IAM and STS.
7. https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_iam-condition-keys.html : OIDC condition keys.
8. https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_elements_condition_operators.html : string operators, missing keys.
9. https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html : temporary credentials, least privilege.
10. https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html : roles.
11. https://learn.microsoft.com/en-us/azure/devops/pipelines/library/connect-to-azure?view=azure-devops : service connections (background).